

De Web Services à Comunicação Indireta em um Sistema de Controle de Academia

Murilo Fragoso e Pedro Erykles

Junho 2026

1 Introdução

Este relatório documenta duas fases evolutivas do projeto: a implementação de uma API RESTful para comunicação cliente-servidor (atendendo ao Trabalho 3) e a posterior evolução para um modelo de comunicação indireta utilizando um sistema publicar-assinar baseado em eventos, garantindo desacoplamento espacial e temporal completo (atendendo ao Trabalho 4).

2 Implementação de Web Services e API

2.1 Objetos Distribuídos e Servidor

O servidor, implementado em Java com Spring Boot, gerencia e expõe, no mínimo, três objetos de domínio distribuídos que transitam entre cliente e servidor e são persistidos no Redis, que foi utilizado como banco de dados e, posteriormente, como *broker* de mensagens:

1. **Aluno:** Representa o aluno da academia, contendo dados pessoais, matrícula e seu cronograma de treinos. Os serviços disponíveis incluem listagem, cadastro, busca por matrícula e remoção de registros.
2. **TreinoDiario:** Representa a rotina de um dia específico de um Aluno, contendo o grupo muscular e uma lista de exercícios. Permite a criação e a consulta de treinos associados a um aluno.
3. **Exercicio:** Entidade que encapsula a menor unidade de um treino, contendo nome, séries, repetições, descanso e carga. Permite criar um exercício que será adicionado a lista de exercícios de um aluno. Permite criar novos exercícios para compor a lista de atividades do aluno.

Essas classes implementam a interface *Serializable* e são mapeadas para DTOs (Data Transfer Objects) no controller, garantindo que apenas os dados necessários sejam expostos via API.

2.2 Interoperabilidade e Clientes Heterogêneos

Um dos requisitos centrais do Trabalho 3 é a interação com clientes implementados em linguagens diferentes da utilizada no servidor (Java). Para demonstrar a interoperabilidade da API REST, foram desenvolvidos dois clientes distintos em projetos separados:

- **Cliente em Python:** Utiliza a biblioteca `requests` para consumir os endpoints, demonstrando a facilidade de integração da API com linguagens amplamente usadas em automação e scripts.
- **Cliente em JavaScript:** Utiliza a biblioteca `axios` para realizar as requisições HTTP, o intuito é simular um cenário real de integração com aplicações Web ou *back-ends* em Node.js.

Ambos os clientes se comunicam exclusivamente via requisições HTTP (GET, POST, PUT, DELETE) para o servidor Spring Boot, provando que o acoplamento de protocolo (HTTP/JSON) substituiu com sucesso o acoplamento de linguagem e plataforma.

3 Evolução para Comunicação Indireta

Embora a API REST resolva a comunicação cliente-servidor, a lógica interna do servidor ainda possuía acoplamento direto. Para resolver isso, escolhemos implementar um sistema do tipo Publicar-Assinar (*Publish-Subscribe*), implementada através de Redis Streams, separando a aplicação em dois projetos independentes: a `gym-api` (Publicador) e a `gym-worker` (Assinante).

3.1 Justificativa da Escolha

A abordagem Pub-Sub com Redis Streams foi escolhida porque, além de já estarmos utilizando o Redis no projeto, um sistema de academia possui múltiplos módulos que reagem a ações de domínio (ex: cadastro de aluno, criação de treino). Diferente do Pub/Sub clássico do Redis (que é *fire-and-forget* e perde mensagens se o receptor cair), o Redis Streams atua como um *log* de eventos persistente. Isso garante o desacoplamento temporal completo, exigido no trabalho: se o módulo de notificações (*Worker*) estiver fora do ar, a mensagem aguarda na fila do Redis até que ele retorne, sem bloquear a API principal.

3.2 Mapeamento Teórico

- `publish(e)`: materializado pelo método `GymEventPublisher.publishEvent()`. O `GestaoTreinoService` cria o evento e (ex: `AlunoCadastradoEvent`), serializa em JSON e o envia ao *broker* Redis.
- `subscribe(f)`: realizado em duas camadas. O `StreamMessageListenerContainer` assina o tópico `"gym-events-stream"`. O filtro `f` é aplicado no nível de conteúdo dentro dos métodos `process()` dos *Subscribers*, que verificam o `eventType` (ex: `if "AlunoCadastradoEvent".equals(...)`) para processar apenas eventos de seu interesse.

- `notify(e)`: executado assincronamente pelo *container* do Spring. Ao detectar uma nova mensagem no *stream*, o *framework* invoca o método `process(MapRecord)` do componente anotado com `@Component`, entregando o evento.
- `unsubscribe(f)`: gerenciado pelo ciclo de vida da aplicação. Interromper a execução do *Worker* equivale a revogar o interesse, sem afetar a operação do publicador (API).

4 Análise de Impacto

4.1 Overhead introduzido

A introdução de um intermediário (*Broker* Redis) adiciona flexibilidade arquitetural, mas também gera sobrecarga (*overhead*) e complexidade de gerenciamento.

1. **Latência de Serialização:** O objeto Java precisa ser convertido para JSON (via `ObjectMapper`) antes de ser enviado ao Redis, e desserializado no receptor.
2. **Latência de Rede e *Polling*:** Há uma chamada de rede extra para gravar no Redis. Além disso, o *listener* do Spring utiliza um mecanismo de *polling* configurado com `pollTimeout` de 100ms para verificar novas mensagens, gerando um consumo contínuo, ainda que mínimo, de CPU.

4.2 Estratégias de Redução

- **Velocidade *In-Memory*:** Como o Redis opera em memória, a latência de escrita/leitura do *stream* é da ordem de sub-milissegundos, tornando o *overhead* de rede praticamente imperceptível para o usuário final.
- **Processamento Assíncrono Real:** A resposta HTTP ao cliente (via Swagger, Python ou JavaScript) permanece rápida, pois a publicação no *stream* é uma operação de "disparar e esquecer" do ponto de vista da *thread* principal da API. O processamento pesado ocorre em background no *Worker*.

5 Conclusão

A transição para Web Services (exigido no Trabalho 3) garantiu a interoperabilidade entre sistemas heterogêneos, nesse caso, entre sistemas implementados em Java, JavaScript e Python. A posterior introdução do Redis Streams (utilizado para o Trabalho 4) eliminou o acoplamento ponto a ponto na camada de domínio, conferindo ao sistema resiliência a falhas temporárias de módulos secundários e facilidade para a adição de novas funcionalidades sem refatoração do código principal.

É importante ressaltar que ainda que tenha sido introduzido um *overhead* de desempenho, ele se torna irrelevante e amplamente justificado pelos ganhos em manutenibilidade, escalabilidade e robustez distribuída.